

Guía paso a paso para el Desarrollo Frontend con HTML y JavaScript: Consumo de APIs REST.

Docente: Daniel Alfonso Martínez Payán.

Duración aproximada: 6 horas.

Introducción

Para desarrollar aplicaciones web modernas del lado del cliente, según Esquivel Paula, Quisaguano Collaguazo, Caluña Guaman y Llambo Alvarez (2024), es fundamental estructurar correctamente la interfaz de usuario y seleccionar tecnologías adecuadas para lograr una experiencia fluida, accesible y que pueda mantenerse. HTML, CSS y JavaScript siguen siendo las tecnologías esenciales para la construcción de interfaces, y su correcta integración permite un desarrollo más organizado y escalable.

En este proyecto básico, se abordará la creación de una interfaz web utilizando HTML para la estructura, CSS para el diseño visual y JavaScript para la interacción dinámica. El objetivo principal es desarrollar una aplicación capaz de consumir una API REST, mostrando información dinámica y permitiendo operaciones CRUD (crear, leer, actualizar y eliminar) desde el frontend. Según Esquivel Paula, Quisaguano Collaguazo, Caluña Guaman y Llambo Alvarez (2024), mantener un enfoque modular y limpio en el desarrollo del frontend facilita la reutilización de componentes y mejora la mantenibilidad del código a largo plazo.

Configuración Adicional

La arquitectura modular adoptada en este proyecto facilita su escalabilidad del lado del cliente. Siguiendo principios similares al patrón de diseño MVC (Modelo, Vista, Controlador), se organiza el código en componentes separados para la estructura (HTML), el diseño visual (CSS) y la lógica de interacción (JavaScript). Tal y como mencionan Haro, Guarda, Zambrano Peñaherrera y Ninahualpa Quiña (2019), esta separación de responsabilidades no solo permite una expansión ordenada del proyecto, sino que también facilita la integración de nuevas funcionalidades sin comprometer la estructura existente, mejorando la mantenibilidad del código frontend.

Objetivo

El propósito de este proyecto es proporcionar a los estudiantes una comprensión sólida de las herramientas y metodologías fundamentales para desarrollar interfaces web dinámicas utilizando HTML, CSS y JavaScript. A través de la correcta estructuración de componentes visuales y la implementación de funciones para el consumo de APIs REST, los estudiantes adquirirán los conocimientos necesarios para construir interfaces escalables, accesibles y funcionales que interactúan eficientemente con sistemas backend.

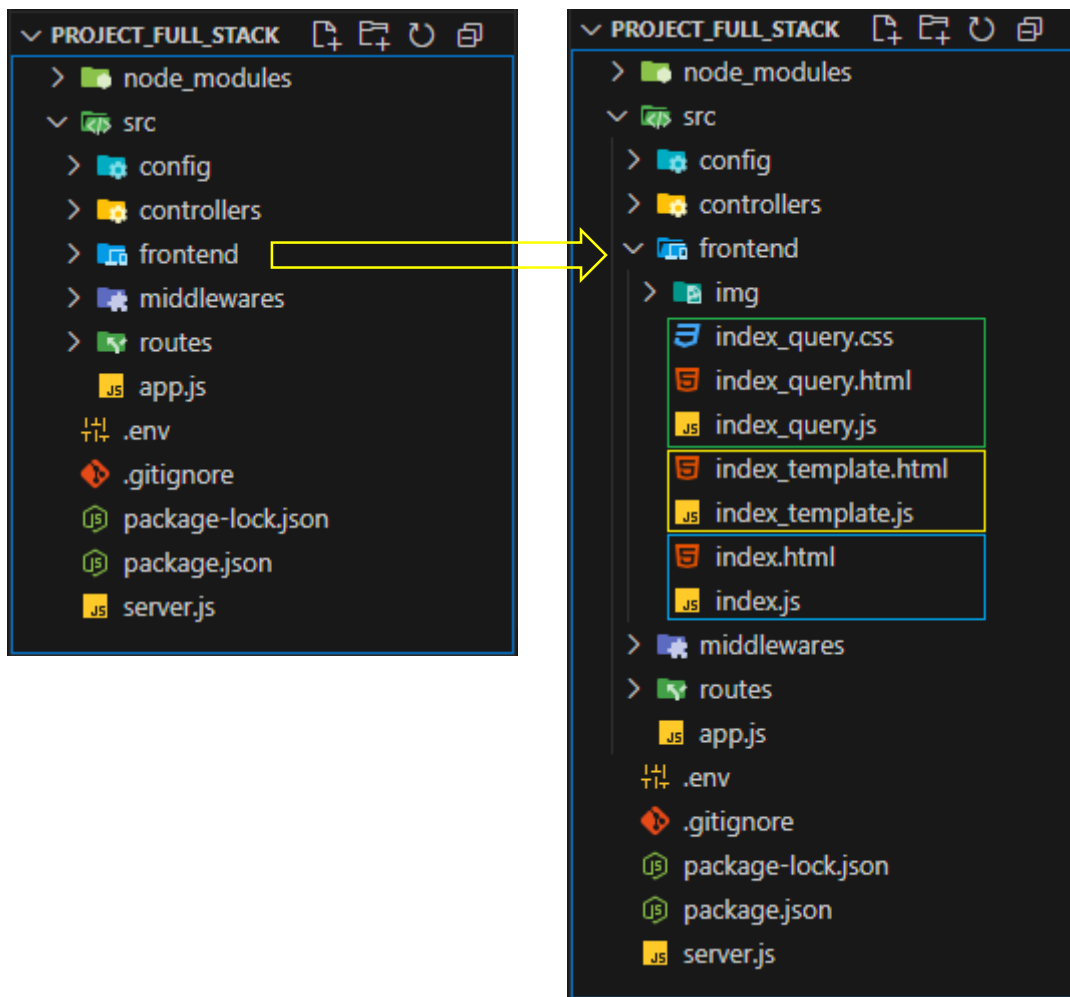
Contenido

Guía paso a paso para el Desarrollo Frontend con HTML y JavaScript: Consumo de APIs REST	1
Introducción.....	1
Estructura de carpetas del proyecto Frontend	3
Realizar maquetado en el archivo index.html en VSC	4
Configuración del archivo index.js (JavaScript) e integración con la estructura HTML. ..	6
Realizar maquetado en el archivo index_template.html en VSC (Segunda Forma)	8
Configuración del archivo index_template.js (JavaScript) e integración con la estructura HTML.	10
Maquetado en el archivo index_query.html e interacción posterior con CSS y JavaScript mediante querySelector (Tercera Forma)	12
Configuración del archivo index_query.js (JavaScript) e integración con la estructura HTML mediante querySelector.	14
Referencias bibliográficas	19

Estructura de carpetas del proyecto Frontend

Cree una carpeta vacía con el nombre frontend. Dentro de esta carpeta, genere los siguientes archivos básicos con los nombres sugeridos: index.html e index.js, los cuales estarán vinculados entre sí para representar la estructura y funcionalidad principal de la interfaz. Adicionalmente, puede agregar otros archivos organizados según su propósito, utilizando convenciones como template.js para manejar plantillas HTML desde JavaScript o query.js para gestionar las peticiones a la API. Esta organización modular facilita el mantenimiento del código y la escalabilidad del proyecto.

Ilustración 1. Estructura Básica



Realizar maquetado en el archivo index.html en VSC

El maquetado es el primer paso para construir una interfaz web organizada y funcional. En esta etapa, se define la estructura visual de la página utilizando HTML, estableciendo los bloques principales que compondrán la aplicación, como encabezados, menús, formularios, botones y secciones de contenido.


```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="UTF-8">
5      <!-- Define el tipo de codificación de caracteres -->
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <!-- Configura la visualización en dispositivos móviles -->
8      <title>CRUD de Personas e Imágenes</title>
9      <!-- Título de la página en la pestaña del navegador -->
10 </head>
11 <body>
12     <h1>Gestión de Personas</h1>
13
14     <form id="personaForm">
15         <!-- Formulario para crear o editar personas -->
16
17         <input type="hidden" id="id_persona">
18         <!-- Campo oculto para almacenar el ID cuando se edita una persona -->
19
20         <!--
21         Contenedor <div> utilizado para agrupar el label y el input,
22         lo cual permite aplicar márgenes, estilos o estructuras en bloque fácilmente.
23         El <label> proporciona un texto descriptivo para el campo y se asocia al input mediante el atributo 'for',
24         mejorando la accesibilidad y la usabilidad del formulario.
25         -->
26         <div>
27             <label for="nombre">Nombre:</label>
28             <input type="text" id="nombre" required>
29         </div>
30
31         <div>
32             <label for="apellido">Apellido:</label>
33             <input type="text" id="apellido" required>
34         </div>
35
36         <div>
37             <label for="tipo_identificacion">Tipo de Identificación:</label>
38             <select id="tipo_identificacion" required>
39                 <option value="CC">Cédula de Ciudadanía</option>
40                 <option value="TI">Tarjeta de Identidad</option>
41                 <option value="CE">Cédula de Extranjería</option>
42             </select>
43         </div>
44
45         <div>
46             <label for="nuip">Número de Identificación:</label>
47             <input type="number" id="nuip" required>
48         </div>
49
50         <div>
51             <label for="email">Email:</label>
52             <input type="email" id="email" required>
53         </div>
54
55         <div>
56             <label for="clave">Contraseña:</label>
57             <input type="password" id="clave" required>
58         </div>
59
60         <div>
61             <label for="salario">Salario:</label>
62             <input type="number" id="salario" step="0.01" required>
63         </div>
64
65         <div>
66             <label for="activo">Activo:</label>
67             <input type="checkbox" id="activo">
68         </div>
69
70         <!-- Sección para manejo de imágenes -->
71         <div>
72             <label for="imagen">Imagen:</label>
73             <input type="file" id="imagen" accept="image/*">
74             <img id="previewImagen" style="max-width: 200px; display: none;">
75         </div>
76
77         <button type="submit" id="btnGuardar">Guardar</button>
78         <button type="button" id="btnCancelar">Cancelar</button>
79     </form>
80
81     <!-- Tabla para mostrar las personas -->
82     <table id="tablaPersonas" style="width: 100%; margin-top: 20px; border-collapse: collapse;">
83         <thead>
84             <tr>
85                 <th style="border: 1px solid #ddd; text-align: center; padding: 8px;">ID</th>
86                 <th style="border: 1px solid #ddd; text-align: center; padding: 8px;">Nombre</th>
87                 <th style="border: 1px solid #ddd; text-align: center; padding: 8px;">Apellido</th>
88                 <th style="border: 1px solid #ddd; text-align: center; padding: 8px;">Email</th>
89                 <th style="border: 1px solid #ddd; text-align: center; padding: 8px;">Imagen</th>
90                 <th style="border: 1px solid #ddd; text-align: center; padding: 8px;">Acciones</th>
91             </tr>
92         </thead>
93         <!--
94         <thead> define la cabecera de la tabla, utilizada para describir el contenido de cada columna.
95         Cada <th> representa un encabezado de columna, con estilos aplicados para mejorar la presentación:
96         - Borde fino y gris claro alrededor de cada celda.
97         - Alineación centrada del texto.
98         - Espaciado interno (padding) para una mejor legibilidad.
99         Las columnas muestran: ID, Nombre, Apellido, Email, Imagen asociada y las acciones disponibles (como editar o eliminar).
100         -->
101
102         <tbody id="tablaPersonasBody">
103             <!-- Los datos se cargarán dinámicamente -->
104         </tbody>
105     </table>
106
107     <script src="index.js"></script>
108     <!--
109     Inclusión del archivo JavaScript externo 'index.js'.
110     Este script contiene la lógica del CRUD (Crear, Leer, Actualizar y Eliminar), así como la interacción con el formulario y la tabla.
111     Al cargarlo al final del body, se asegura que el DOM esté completamente disponible cuando el script se ejecute.
112     -->
113 </body>
114 </html>
```

Configuración del archivo index.js (JavaScript) e integración con la estructura HTML.

Una vez maquetada la interfaz en el archivo index.html, el siguiente paso es incorporar funcionalidad mediante JavaScript. Para ello, se configura el archivo index.js, que contendrá el código necesario para interactuar dinámicamente con los elementos definidos en el HTML. Esta integración permite capturar eventos del usuario, modificar el contenido de la página en tiempo real y conectar la interfaz con servicios externos como APIs. Este enfoque modular mejora la organización del proyecto y facilita la escalabilidad del código frontend.

Realizar maquetado en el archivo index_template.html en VSC (Segunda Forma)

El archivo index_template.html representa una segunda forma de estructurar la interfaz de la aplicación web. En este enfoque, se utiliza una plantilla HTML base que puede ser reutilizada y adaptada para múltiples vistas o componentes. Este tipo de maquetado permite mantener una organización más clara del código, especialmente cuando se trabaja con aplicaciones que requieren modularidad o integración posterior con JavaScript. El uso de un template facilita la separación entre la estructura visual y los datos dinámicos, permitiendo generar contenido automáticamente a partir de información proveniente de una API u otras fuentes. La estructura del archivo incluirá elementos fundamentales como encabezados, menús, formularios y botones, que servirán como base para el desarrollo de la lógica y la funcionalidad dinámica del sistema.

Configuración del archivo index_template.js (JavaScript) e integración con la estructura HTML.

Una vez definida la estructura base en el archivo index_template.html, el siguiente paso es agregar la funcionalidad mediante el archivo index_template.js. Este archivo JavaScript se encargará de manipular dinámicamente la plantilla HTML, insertando datos, generando elementos visuales de forma programada y gestionando eventos del usuario. La integración entre index_template.js y el HTML permite reutilizar estructuras, cargar contenido desde APIs y actualizar la interfaz sin necesidad de recargar la página. Este enfoque mejora la modularidad del proyecto y es especialmente útil para desarrollar aplicaciones web escalables y dinámicas.


```
src > frontend > index_template.js > mostrarPersonas
1 // Variables globales
2 const API_URL = 'http://localhost:3000/api'; // URL base de la API backend
3 let personas = []; // Arreglo donde se almacenan las personas obtenidas del servidor
4
5 // Elementos del DOM
6 const personaForm = document.getElementById('personaForm'); // Formulario principal
7 const tablaPersonasBody = document.getElementById('tablaPersonasBody'); // Cuerpo de la tabla donde se listan
  personas
8 const btnCancelar = document.getElementById('btnCancelar'); // Botón para limpiar el formulario
9 const imagenInput = document.getElementById('imagen'); // Input de imagen
10 const previewImagen = document.getElementById('previewImagen'); // Imagen para previsualizar la subida
11
12 // Event Listeners
13 document.addEventListener('DOMContentLoaded', cargarPersonas); // Carga personas al iniciar la página
14 personaForm.addEventListener('submit', manejarSubmit); // Enviar el formulario
15 btnCancelar.addEventListener('click', limpiarFormulario); // Botón de cancelar limpia el formulario
16 imagenInput.addEventListener('change', manejarImagen); // Cargar previsualización cuando se selecciona imagen
17
18 // Función que obtiene personas del backend
19 async function cargarPersonas() {
20   try {
21     const response = await fetch(`${API_URL}/personas`); // Solicitud GET a la API
22     personas = await response.json(); // Almacena respuesta en arreglo
23     await mostrarPersonas(); // Muestra las personas en la tabla
24   } catch (error) {
25     console.error('Error al cargar personas:', error);
26   }
27 }
28
29 // Función para mostrar todas las personas en la tabla
30 async function mostrarPersonas() {
31   // Limpia el contenido actual del cuerpo de la tabla para evitar duplicados
32   tablaPersonasBody.innerHTML = '';
33
34   // Obtiene el elemento <template> que contiene la estructura de una fila de persona
35   const template = document.getElementById('template');
36
37   // Recorre la lista de personas obtenidas desde el backend
38   for (const persona of personas) {
39     // Clona el contenido del template (la fila predefinida)
40     const clone = template.content.cloneNode(true);
41
42     // Obtiene todas las celdas <td> dentro del clon
43     const tds = clone.querySelectorAll('td');
44
45     // Inicializa el contenido de imagen como 'Sin imagen' por defecto
46     let imagenHTML = 'Sin imagen';
47
48     // Intenta obtener la imagen de la persona desde el backend
49     try {
50       // Realiza una petición GET al endpoint de imagen de la persona por su ID
51       const response = await fetch(`${API_URL}/imagenes/obtener/personas/id_persona/${persona.id_persona}`);
52
53       // Convierte la respuesta en un objeto JSON
54       const data = await response.json();
55
56       // Si hay una imagen en la respuesta, se construye la etiqueta <img> con la imagen en base64
57       if (data.imagen) {
58         imagenHTML = ``;
60       }
61     } catch (error) {
62       // Si ocurre un error al obtener la imagen, lo muestra en consola
63       console.error('Error al cargar imagen:', error);
64     }
65
66     // Llena las celdas con los datos de la persona
67     tds[0].textContent = persona.id_persona; // ID
68     tds[1].textContent = persona.nombre; // Nombre
69     tds[2].textContent = persona.apellido; // Apellido
70     tds[3].textContent = persona.email; // Email
71     tds[4].innerHTML = imagenHTML; // Imagen (si existe, muestra imagen, si no, "Sin imagen")
72
73     // Busca los botones de editar y eliminar dentro del clon
74     const btnEditar = clone.querySelector('.btn-editar');
75     const btnEliminar = clone.querySelector('.btn-eliminar');
76
77     // Asigna el evento de clic al botón de editar, llamando a la función con el ID de la persona
78     btnEditar.addEventListener('click', () => editarPersona(persona.id_persona));
79
80     // Asigna el evento de clic al botón de eliminar, llamando a la función con el ID de la persona
81     btnEliminar.addEventListener('click', () => eliminarPersona(persona.id_persona));
82
83     // Finalmente, agrega la fila clonada (con datos y botones configurados) al cuerpo de la tabla
84     tablaPersonasBody.appendChild(clone);
85   }
86 }
87
88 // Función que maneja el envío del formulario (crear o editar persona)
89 async function manejarSubmit(e) {
90   e.preventDefault(); // Previene el comportamiento por defecto del formulario
91
92   // Obtiene los datos del formulario
93   const persona = {
94     id_persona: document.getElementById('id_persona').value || null,
95     nombre: document.getElementById('nombre').value,
96     apellido: document.getElementById('apellido').value,
97     tipo_identificacion: document.getElementById('tipo_identificacion').value,
98     nuip: parseInt(document.getElementById('nuip').value),
99     email: document.getElementById('email').value,
100     clave: document.getElementById('clave').value,
101     salario: parseFloat(document.getElementById('salario').value),
102     activo: document.getElementById('activo').checked
103   };
104
105   try {
106     if (persona.id_persona) {
107       // Si estamos editando (id_persona existe)
108
109       // Subir imagen si fue seleccionada
110       if (imagenInput.files[0]) {
111         const imagenBase64 = await convertirImagenABase64(imagenInput.files[0]);
112         await fetch(`${API_URL}/imagenes/subir/personas/id_persona/${persona.id_persona}`, {
113           method: 'PUT',
114           headers: { 'Content-Type': 'application/json' },
115           body: JSON.stringify({ imagen: imagenBase64 })
116         });
117       }
118       // Actualizar los datos de la persona
119       await actualizarPersona(persona);
120     } else {
121       // Si es nueva persona
122       const nuevaPersona = await crearPersona(persona); // Crear persona
123       if (imagenInput.files[0]) {
124         const imagenBase64 = await convertirImagenABase64(imagenInput.files[0]);
125         await fetch(`${API_URL}/imagenes/insertar/personas/id_persona/${nuevaPersona.id}`, {
126           method: 'POST',
127           headers: { 'Content-Type': 'application/json' },
128           body: JSON.stringify({ imagen: imagenBase64 })
129         });
130       }
131     }
132     limpiarFormulario(); // Limpia el formulario
133     cargarPersonas(); // Recarga la lista
134   } catch (error) {
135     console.error('Error al guardar persona:', error);
136     alert('Error al guardar los datos: ' + error.message);
137   }
138 }
139
140 // Crea una persona nueva en la base de datos
141 async function crearPersona(persona) {
142   const response = await fetch(`${API_URL}/personas`, {
143     method: 'POST',
144     headers: { 'Content-Type': 'application/json' },
145     body: JSON.stringify(persona)
146   });
147   return await response.json(); // Devuelve el objeto persona nuevo con id
148 }
149
150 // Actualiza una persona existente
151 async function actualizarPersona(persona) {
152   const response = await fetch(`${API_URL}/personas/${persona.id_persona}`, {
153     method: 'PUT',
154     headers: { 'Content-Type': 'application/json' },
155     body: JSON.stringify(persona)
156   });
157   return await response.json();
158 }
159
160 // Elimina persona (y su imagen si existe)
161 async function eliminarPersona(id) {
162   if (confirm('¿Está seguro de eliminar esta persona?')) {
163     try {
164       await fetch(`${API_URL}/imagenes/eliminar/personas/id_persona/${id}`, { method: 'DELETE' }); // Elimina imagen
165       await fetch(`${API_URL}/personas/${id}`, { method: 'DELETE' }); // Elimina persona
166       cargarPersonas(); // Recarga la lista
167     } catch (error) {
168       console.error('Error al eliminar persona:', error);
169       alert('Error al eliminar la persona: ' + error.message);
170     }
171   }
172 }
173
174 // Llena el formulario con los datos de una persona para editar
175 async function editarPersona(id) {
176   const persona = personas.find(p => p.id_persona === id);
177   if (persona) {
178     document.getElementById('id_persona').value = persona.id_persona;
179     document.getElementById('nombre').value = persona.nombre;
180     document.getElementById('apellido').value = persona.apellido;
181     document.getElementById('tipo_identificacion').value = persona.tipo_identificacion;
182     document.getElementById('nuip').value = persona.nuip;
183     document.getElementById('email').value = persona.email;
184     document.getElementById('clave').value = ''; // No se muestra la contraseña
185     document.getElementById('salario').value = persona.salario;
186     document.getElementById('activo').checked = persona.activo;
187
188     // Carga la imagen si existe
189     try {
190       const response = await fetch(`${API_URL}/imagenes/obtener/personas/id_persona/${id}`);
191       const data = await response.json();
192       if (data.imagen) {
193         previewImagen.src = `data:image/jpeg;base64,${data.imagen}`;
194         previewImagen.style.display = 'block';
195       } else {
196         previewImagen.style.display = 'none';
197         previewImagen.src = '';
198       }
199     } catch (error) {
200       console.error('Error al cargar imagen:', error);
201       previewImagen.style.display = 'none';
202       previewImagen.src = '';
203     }
204   }
205 }
206
207 // Limpia todos los campos del formulario
208 function limpiarFormulario() {
209   personaForm.reset();
210   document.getElementById('id_persona').value = '';
211   previewImagen.style.display = 'none';
212   previewImagen.src = '';
213 }
214
215 // Muestra una previsualización de la imagen seleccionada
216 function manejarImagen(e) {
217   const file = e.target.files[0];
218   if (file) {
219     const reader = new FileReader();
220     reader.onload = function(e) {
221       previewImagen.src = e.target.result;
222       previewImagen.style.display = 'block';
223     }
224     reader.readAsDataURL(file);
225   } else {
226     previewImagen.style.display = 'none';
227     previewImagen.src = '';
228   }
229 }
230
231 // Convierte imagen a base64 para enviarla al backend
232 function convertirImagenABase64(file) {
233   return new Promise((resolve, reject) => {
234     const reader = new FileReader();
235     reader.readAsDataURL(file);
236     reader.onload = () => {
237       const base64 = reader.result.split(',')[1]; // Elimina el prefijo del data URI
238       resolve(base64);
239     };
240     reader.onerror = error => reject(error);
241   });
242 }
243
```


Maquetado en el archivo index_query.html e interacción posterior con CSS y JavaScript mediante querySelector (Tercera Forma)

El archivo index_query.html representa una alternativa para estructurar la interfaz de la aplicación web, diseñada específicamente para ser manipulada a través de hojas de estilo (index_query.css) y scripts JavaScript (index_query.js) utilizando el método querySelector. Este enfoque permite seleccionar y modificar de forma precisa los elementos del DOM con base en sus identificadores, clases o etiquetas. El maquetado establece una estructura clara y funcional que incluye componentes como encabezados, formularios, botones y contenedores de contenido. Esta forma de trabajo facilita el desarrollo de interfaces interactivas y responsivas, promoviendo la separación entre diseño, estructura y lógica, y ofreciendo mayor control sobre los elementos visuales y su comportamiento dinámico.

Estilos para el maquetado en el archivo index_query.css

El archivo index_query.css se encarga de definir los estilos visuales que darán forma y coherencia a la interfaz maquetada en el archivo index_query.html. En este archivo se aplican reglas de estilo para elementos como encabezados, formularios, botones, contenedores y secciones, asegurando una presentación atractiva y una experiencia de usuario clara e intuitiva. Es importante destacar que este diseño visual se complementará con la lógica funcional contenida en el archivo index_query.js, el cual utilizará querySelector para manipular dinámicamente los elementos definidos en el HTML, logrando así una interacción completa entre estructura, estilo y comportamiento del sistema.

src > frontend > index_query.css > ...

```
1  /* Estilos generales */
2  body {
3      font-family: Arial, sans-serif;
4      margin: 20px;
5  }
6
7  h1 {
8      color: #333;
9      margin-bottom: 20px;
10 }
11
12 /* Estilos del formulario */
13 #personaForm {
14     margin-bottom: 20px;
15 }
16
17 #personaForm div {
18     margin-bottom: 10px;
19 }
20
21 #personaForm label {
22     display: inline-block;
23     width: 150px;
24 }
25
26 #personaForm input[type="text"],
27 #personaForm input[type="email"],
28 #personaForm input[type="number"],
29 #personaForm input[type="password"],
30 #personaForm select {
31     width: 200px;
32     padding: 5px;
33 }
34
35 #personaForm button {
36     margin-right: 10px;
37     padding: 5px 15px;
38 }
39
40 /* Estilos de la tabla */
41 #tablaPersonas {
42     width: 100%;
43     margin-top: 20px;
44     border-collapse: collapse;
45 }
46
47 #tablaPersonas th,
48 #tablaPersonas td {
49     border: 1px solid #ddd;
50     text-align: center;
51     padding: 8px;
52 }
53
54 #tablaPersonas th {
55     background-color: #f2f2f2;
56 }
57
58 /* Estilos de la imagen */
59 #previewImagen {
60     max-width: 200px;
61     display: none;
62 }
63
64 /* Estilos de los botones */
65 .btn-editar {
66     background-color: #4CAF50;
67     color: white;
68     border: none;
69     padding: 5px 10px;
70     margin-right: 5px;
71     cursor: pointer;
72 }
73
74 .btn-eliminar {
75     background-color: #f44336;
76     color: white;
77     border: none;
78     padding: 5px 10px;
79     cursor: pointer;
80 }
81
82 /* Estilos para las imágenes en la tabla */
83 #tablaPersonas img {
84     max-width: 100px;
85     max-height: 100px;
86 }
```

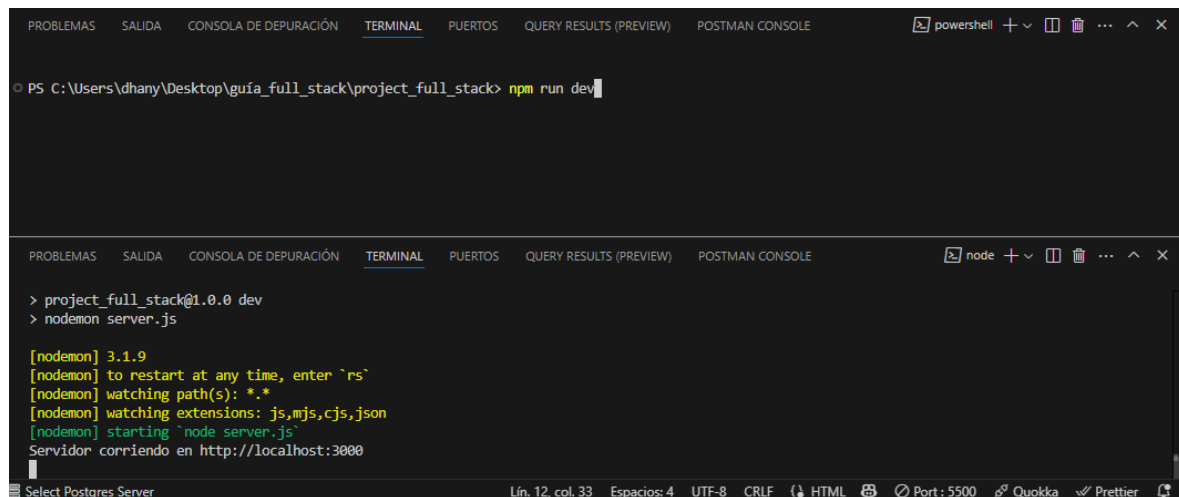
Configuración del archivo index_query.js (JavaScript) e integración con la estructura HTML mediante querySelector.

Una vez definida la estructura base en el archivo index_query.html, el siguiente paso es incorporar la funcionalidad mediante el archivo index_query.js. Este archivo JavaScript utilizará el método querySelector para seleccionar y manipular dinámicamente los elementos del DOM definidos en el HTML. A través de este método, se pueden insertar datos, modificar el contenido de los elementos, aplicar cambios de estilo desde el archivo index_query.css y gestionar eventos de usuario de manera eficiente. La integración entre index_query.js y index_query.html permite crear una interfaz más interactiva y responsive, sin necesidad de recargar la página. Este enfoque modular facilita la escalabilidad del proyecto y la implementación de características dinámicas.


```
index_query.js X
src > frontend > index_query.js > resetearFormulario
1 // =====
2 // VARIABLES GLOBALES
3 // =====
4
5 // URL base de la API
6 const API_URL = 'http://localhost:3000/api';
7
8 // Arreglo donde se almacenarán las personas traídas desde la API
9 let personas = [];
10
11 // Variable para determinar si estamos editando o creando
12 let modoEdicion = false;
13
14 // =====
15 // ELEMENTOS DEL DOM
16 // =====
17
18 // Referencia al formulario de personas
19 const form = document.querySelector('#personaForm');
20
21 // Cuerpo de la tabla donde se insertarán las filas dinámicamente
22 const tablaBody = document.querySelector('#tablaPersonasBody');
23
24 // Template HTML para generar filas de tabla
25 const template = document.querySelector('#template');
26
27 // Botón para guardar (crear o actualizar)
28 const btnGuardar = document.querySelector('#btnGuardar');
29
30 // Botón para cancelar la edición
31 const btnCancelar = document.querySelector('#btnCancelar');
32
33 // Input de imagen y su previsualización
34 const inputImagen = document.querySelector('#imagen');
35 const previewImagen = document.querySelector('#previewImagen');
36
37 // =====
38 // CAMPOS DEL FORMULARIO
39 // =====
40
41 const campos = {
42   id: document.querySelector('#id_persona'),
43   nombre: document.querySelector('#nombre'),
44   apellido: document.querySelector('#apellido'),
45   tipo_identificacion: document.querySelector('#tipo_identificacion'),
46   nuip: document.querySelector('#nuip'),
47   email: document.querySelector('#email'),
48   clave: document.querySelector('#clave'),
49   salario: document.querySelector('#salario'),
50   activo: document.querySelector('#activo')
51 };
52
53 // =====
54 // EVENTOS PRINCIPALES
55 // =====
56
57 document.addEventListener('DOMContentLoaded', () => {
58   cargarPersonas(); // Cargar lista inicial
59   form.addEventListener('submit', manejarSubmit); // Guardar datos
60   btnCancelar.addEventListener('click', resetearFormulario); // Cancelar edición
61   inputImagen.addEventListener('change', manejarCambioImagen); // Cargar imagen
62 });
63
64 // =====
65 // FUNCIONES DE LÓGICA
66 // =====
67
68 // Carga todas las personas desde la API
69 async function cargarPersonas() {
70   try {
71     const response = await fetch(`${API_URL}/personas`);
72     personas = await response.json();
73     mostrarPersonas();
74   } catch (error) {
75     console.error('Error al cargar personas:', error);
76   }
77 }
78
79 // Muestra en la tabla todas las personas cargadas
80 async function mostrarPersonas() {
81   tablaBody.innerHTML = ''; // Limpiar tabla
82
83   personas.forEach(async persona => {
84     const clone = template.content.cloneNode(true); // Clonar template
85     const celdas = clone.querySelectorAll('td');
86
87     // Llenar celdas con datos de la persona
88     celdas[0].textContent = persona.id_persona;
89     celdas[1].textContent = persona.nombre;
90     celdas[2].textContent = persona.apellido;
91     celdas[3].textContent = persona.email;
92
93     // Imagen por defecto
94     let imagenHTML = 'Sin imagen';
95
96     try {
97       const response = await fetch(`${API_URL}/imagenes/obtener/personas/id_persona/${persona.id_persona}`);
98       const data = await response.json();
99       if (data.imagen) {
100         imagenHTML = ``;
102       }
103     } catch (error) {
104       console.error('Error al cargar imagen:', error);
105     }
106
107     celdas[4].innerHTML = imagenHTML;
108
109     // Botones de acción
110     const btnEditar = clone.querySelector('.btn-editar');
111     const btnEliminar = clone.querySelector('.btn-eliminar');
112
113     btnEditar.addEventListener('click', () => editarPersona(persona));
114     btnEliminar.addEventListener('click', () => eliminarPersona(persona.id_persona));
115
116     tablaBody.appendChild(clone);
117   });
118 }
119
120 // Manejo del submit del formulario (crear o actualizar persona)
121 async function manejarSubmit(e) {
122   e.preventDefault();
123
124   // Recolectar datos desde el formulario
125   const persona = {
126     nombre: campos.nombre.value,
127     apellido: campos.apellido.value,
128     tipo_identificacion: campos.tipo_identificacion.value,
129     nuip: campos.nuip.value,
130     email: campos.email.value,
131     clave: campos.clave.value,
132     salario: parseFloat(campos.salario.value),
133     activo: campos.activo.checked
134   };
135
136   try {
137     if (modoEdicion) {
138       persona.id_persona = campos.id.value;
139
140       if (inputImagen.files[0]) {
141         const imagenBase64 = await convertirImagenABase64(inputImagen.files[0]);
142         await fetch(`${API_URL}/imagenes/subir/personas/id_persona/${persona.id_persona}`, {
143           method: 'PUT',
144           headers: { 'Content-Type': 'application/json' },
145           body: JSON.stringify({ imagen: imagenBase64 })
146         });
147       }
148
149       await actualizarPersona(persona);
150     } else {
151       const response = await crearPersona(persona);
152
153       if (!response.id) {
154         throw new Error('El servidor no devolvió el ID de la persona creada');
155       }
156
157       if (inputImagen.files[0]) {
158         const imagenBase64 = await convertirImagenABase64(inputImagen.files[0]);
159         await fetch(`${API_URL}/imagenes/insertar/personas/id_persona/${response.id}`, {
160           method: 'POST',
161           headers: { 'Content-Type': 'application/json' },
162           body: JSON.stringify({ imagen: imagenBase64 })
163         });
164       }
165     }
166
167     resetearFormulario();
168     cargarPersonas();
169   } catch (error) {
170     console.error('Error al guardar persona:', error);
171     alert('Error al guardar los datos: ' + error.message);
172   }
173 }
174
175 // Crea una nueva persona en la base de datos
176 async function crearPersona(persona) {
177   const response = await fetch(`${API_URL}/personas`, {
178     method: 'POST',
179     headers: { 'Content-Type': 'application/json' },
180     body: JSON.stringify(persona)
181   });
182
183   if (!response.ok) {
184     throw new Error(`Error HTTP: ${response.status}`);
185   }
186
187   const data = await response.json();
188
189   if (!data.id) {
190     throw new Error('La respuesta del servidor no contiene el ID de la persona');
191   }
192
193   return data;
194 }
195
196 // Actualiza los datos de una persona existente
197 async function actualizarPersona(persona) {
198   const response = await fetch(`${API_URL}/personas/${persona.id_persona}`, {
199     method: 'PUT',
200     headers: { 'Content-Type': 'application/json' },
201     body: JSON.stringify(persona)
202   });
203
204   const data = await response.json();
205   return data;
206 }
207
208 // Elimina una persona y su imagen asociada
209 async function eliminarPersona(id) {
210   if (!confirm('¿Está seguro de eliminar esta persona?')) return;
211
212   try {
213     await fetch(`${API_URL}/imagenes/eliminar/personas/id_persona/${id}`, {
214       method: 'DELETE'
215     });
216     await fetch(`${API_URL}/personas/${id}`, {
217       method: 'DELETE'
218     });
219
220     cargarPersonas(); // Recargar la lista
221   } catch (error) {
222     console.error('Error al eliminar persona:', error);
223     alert('Error al eliminar la persona: ' + error.message);
224   }
225 }
226
227 // Carga los datos de la persona al formulario para editar
228 async function editarPersona(persona) {
229   modoEdicion = true;
230
231   // Cargar campos
232   campos.id.value = persona.id_persona;
233   campos.nombre.value = persona.nombre;
234   campos.apellido.value = persona.apellido;
235   campos.tipo_identificacion.value = persona.tipo_identificacion;
236   campos.nuip.value = persona.nuip;
237   campos.email.value = persona.email;
238   campos.clave.value = persona.clave;
239   campos.salario.value = persona.salario;
240   campos.activo.checked = persona.activo;
241
242   // Cargar imagen
243   try {
244     const response = await fetch(`${API_URL}/imagenes/obtener/personas/id_persona/${persona.id_persona}`);
245     const data = await response.json();
246
247     if (data.imagen) {
248       previewImagen.src = `data:image/jpeg;base64,${data.imagen}`;
249       previewImagen.style.display = 'block';
250     } else {
251       previewImagen.style.display = 'none';
252       previewImagen.src = '';
253     }
254   } catch (error) {
255     console.error('Error al cargar imagen:', error);
256     previewImagen.style.display = 'none';
257     previewImagen.src = '';
258   }
259
260   btnGuardar.textContent = 'Actualizar';
261 }
262
263 // Restablece el formulario al estado original
264 function resetearFormulario() {
265   modoEdicion = false;
266   form.reset();
267   previewImagen.style.display = 'none';
268   previewImagen.src = '';
269   btnGuardar.textContent = 'Guardar';
270 }
271
272 // Previsualiza imagen cuando se selecciona una
273 function manejarCambioImagen(e) {
274   const file = e.target.files[0];
275
276   if (file) {
277     const reader = new FileReader();
278     reader.onload = function(e) {
279       previewImagen.src = e.target.result;
280       previewImagen.style.display = 'block';
281     };
282     reader.readAsDataURL(file);
283   } else {
284     previewImagen.style.display = 'none';
285     previewImagen.src = '';
286   }
287 }
288
289 // Convierte una imagen a Base64 para enviar al servidor
290 function convertirImagenABase64(file) {
291   return new Promise((resolve, reject) => {
292     const reader = new FileReader();
293
294     reader.readAsDataURL(file);
295
296     reader.onload = () => {
297       const base64 = reader.result.split(',')[1]; // Quitar prefijo MIME
298       resolve(base64);
299     };
300
301     reader.onerror = error => reject(error);
302   });
303 }
304
```


Implementación del CRUD con el DOM e inicialización de la terminal del API desde el Frontend: Pruebas y validación.

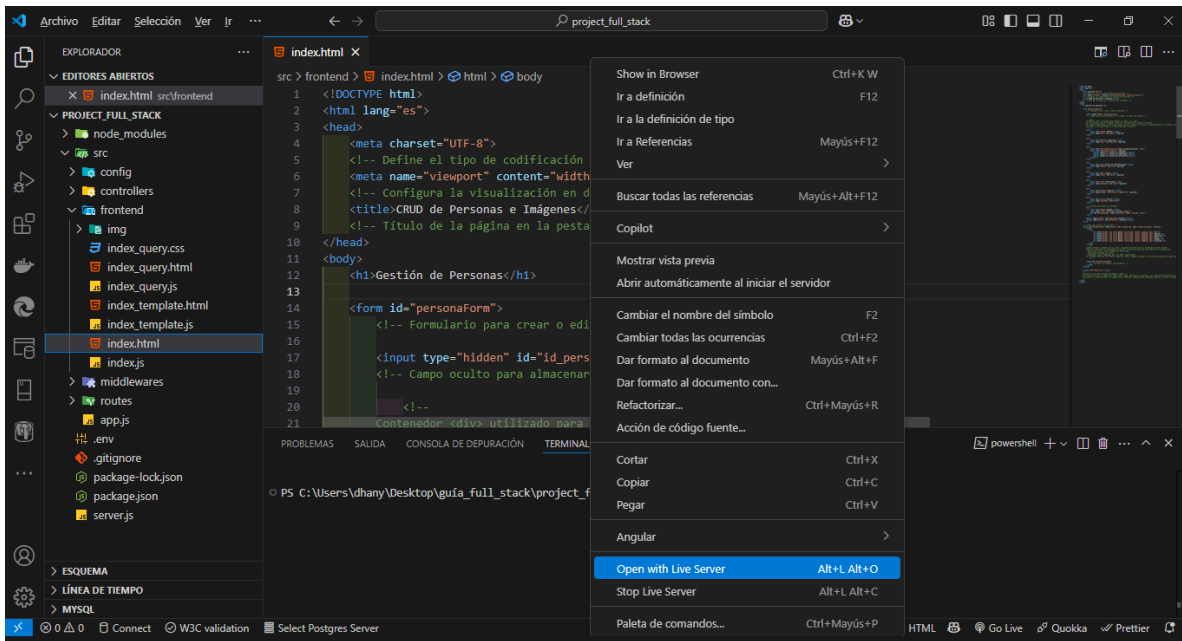
En la guía anterior ([Análisis y Desarrollo de Arquitecturas Backend con Node.js: Enfoques de Seguridad, Escalabilidad y Rendimiento. Una guía paso a paso](#)), se explicó cómo inicializar el proyecto backend utilizando el comando `npm run dev`, lo que permitirá que el servidor backend se ejecute y esté listo para recibir solicitudes desde el frontend. Una vez que el backend esté en funcionamiento, el siguiente paso es iniciar el frontend utilizando la extensión "[Live Server](#)" en Visual Studio Code, con el cual se abrirá el proyecto en el navegador y se podrá interactuar con la interfaz de usuario. Con ambos entornos funcionando, se podrán realizar las pruebas del CRUD con el DOM y verificar la integración con la API, asegurando que el sistema responda correctamente a las solicitudes y manipule los datos como se espera.



```
PS C:\Users\dhany\Desktop\guia_full_stack\project_full_stack> npm run dev

> project_full_stack@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.9
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Servidor corriendo en http://localhost:3000
```

Sin carga de datos desde el servidor

- Ejecutando la aplicación sin archivo cargado: Interacción básica con el DOM
- Aplicación en ejecución: Sin carga de archivo y funcionalidades iniciales
- Vista previa sin carga de datos: Interacción básica con el sistema

Gestión de Personas

Nombre:
Apellido:
Tipo de Identificación:
Número de Identificación:
Email:
Contraseña:
Salario:
Activo: ☐
Imagen: Ningún archivo seleccionado

ID	Nombre	Apellido	Email	Imagen	Acciones
----	--------	----------	-------	--------	----------

Con carga de datos desde el servidor

- Ejecutando la aplicación con carga de archivo: Procesamiento y visualización de datos
- Aplicación en ejecución: Carga de archivo y manipulación de contenido
- Vista avanzada con carga de archivo: Integración de datos y funcionalidades dinámicas.

Gestión de Personas

Nombre:
Apellido:
Tipo de Identificación:
Número de Identificación:
Email:
Contraseña:
Salario:
Activo: ☐
Imagen: Ningún archivo seleccionado

ID	Nombre	Apellido	Email	Imagen	Acciones
7	Stiven Alejandro	Salamanca Ospina	stiven@gmail.com		Editar Eliminar
18	Daniel Alfonso	Martínez Payán	dhanymarth@gmail.com		Editar Eliminar

Gestión de Personas

Nombre:

Apellido:

Tipo de Identificación: Cédula de Ciudadanía ▼

Número de Identificación:

Email:

Contraseña:

Salario:

Activo: ☐

Imagen: Seleccionar archivo Ningún archivo seleccionado

Guardar Cancelar

ID	Nombre	Apellido	Email	Imagen	Acciones
7	Stiven Alejandro	Salamanca Ospina	stiven@gmail.com		Editar Eliminar
18	Daniel Alfonso	Martínez Payán	dhanymarth@gmail.com		Editar Eliminar

Referencias bibliográficas

- Esquivel Paula, G. G., Quisaguano Collaguazo, L. R., Caluña Guaman, A. P., & Llambo Alvarez, S. J. (19 de Noviembre de 2024). Frameworks del lado del Servidor: Caso de Estudio Node JS, Django y Laravel. *593 Digital Publisher CEIT*, 10(1), 403-414. Recuperado el 18 de Abril de 2025, de <https://dialnet.unirioja.es/servlet/articulo?codigo=9966614>
- Haro, E., Guarda, T., Zambrano Peñaherrera, A. O., & Ninahualpa Quiña, G. (2019). Desarrollo backend para aplicaciones web, Servicios Web Restful: Node.js vs Spring Boot. *RISTI. Revista Ibérica de Sistemas e Tecnologias de Informação*(E17), 309-321. Recuperado el 18 de Abril de 2025, de <https://www.proquest.com/openview/a78cfaa62708fd24f38ac8d1025050eb/1?cbl=10063&pq-origsite=gscholar>
- Palmera Quintero, L. M., Ríos Baron, D. J., León, K., & Chinchilla Torres, F. (2024). Desarrollo de un aplicativo móvil con Node.js para la venta de productos agrícolas en MiPymes. *Revista Temario Científico*, 4(2), 1-12. Recuperado el 18 de Abril de 2025, de <https://alinin.org/ojs/index.php/temariocientifico/article/view/157/400>